

Customization of the CVA6 core

Hanan Quispe¹

¹Institut Polytechnique de Paris

March 9, 2026



Outline

- 1 Motivation
- 2 Performance and Resource Usage Measurement
- 3 Experiments
- 4 Results
- 5 Contributions
- 6 Conclusion and Future work

Outline

- 1 Motivation
- 2 Performance and Resource Usage Measurement
- 3 Experiments
- 4 Results
- 5 Contributions
- 6 Conclusion and Future work

The CVA6 Core

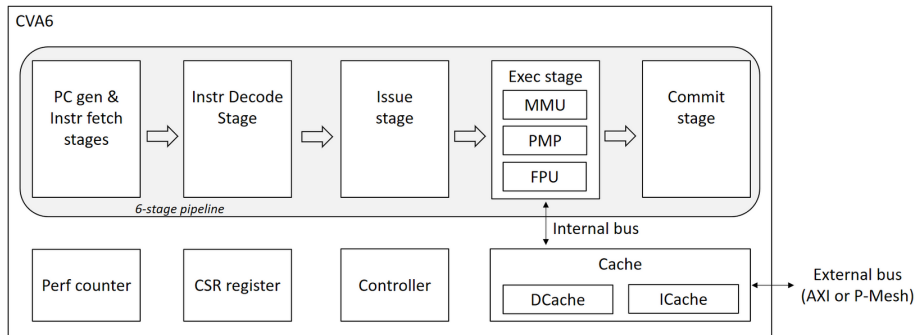
A 6-stage RISC-V Open-Source core

- Industry Standard: CVA6 is a single-issue, in-order design maintained by the OpenHW Foundation (with Thales).
- Versatility: Ideal for ISA extensions (Virtualization), prototyping, and integration into complex multi-core structures like OpenPiton.
- Local Implementation: Internal dual-core prototype developed by the Inria Benagil team for FPGA deployment.



The CVA6 Core

Original Single Core Version



Benagil Team's Dual Core Version



Problem Statement

Problem

The dual-core system is limited by a 1-request-per-cycle bandwidth at the L1D cache.

Research Objectives

- How does the sharing of functional units (starting with the FPU) between two CVA6 cores affect the trade-off between hardware area efficiency and multi-core performance?
- How to measure it accurately.

Project Roadmap

Stage 1 (Baseline Measurements)	Stage 2 (Microarchitectural Study)	Stage 3 (Performance Framework Development)	Stage 4 (Core Modification & Decoupling)	Stage 5 (Arbitration Module Design)	Stage 6 (System Integration & Evaluation)
Deployment of CVA6 on the Pynq Z2	Analysis of Data and Instruction cache sizes	Automated framework for performance measurement	Development of a modified CVA6 version	Development of the shared resource arbitrator	Evaluation of performance and FPGA utilization (FF/LUT/BRAM)
Baseline SoC Utilization measurements (FF/LUT/BRAM)	Study of Branch Predictor and TLB size influence	Integration of NAS Parallel Benchmarks	Implementation of an external module interface [optional]	Implementation of pipeline stalling logic [optional]	Evaluation of performance NPB and Unixbench
Baseline NPB and Unixbench measurements		Integration of UnixBench suite	Enabling connection for FPU management [optional]	Routing of results between multiple cores [optional]	

Outline

- 1 Motivation
- 2 Performance and Resource Usage Measurement
- 3 Experiments
- 4 Results
- 5 Contributions
- 6 Conclusion and Future work

How do We Measure Performance?

What do we measure?

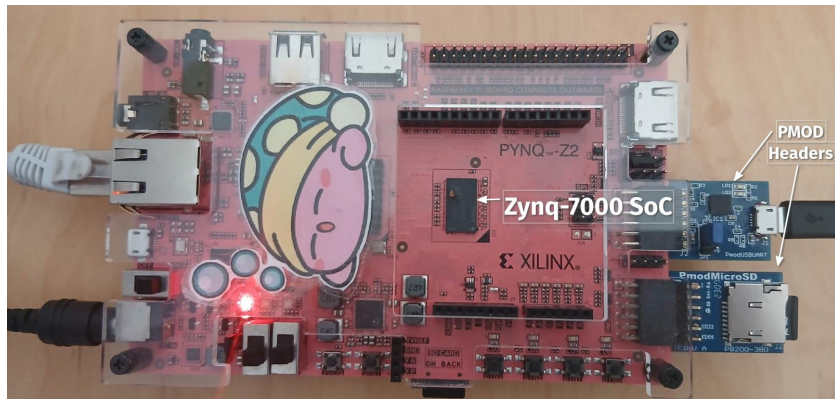
We evaluate performance using well-known workloads: UnixBench and the NAS Parallel Benchmarks (NPB). For each benchmark, we also measure instructions per cycle (*IPC*).

$$IPC = \frac{\text{Number of Instructions}}{\text{Number of cycles}} \quad (1)$$

- Hardware performance counters. *Single Core* available performance counters:
 - Number of Instructions
 - Number of cycles
- Benchmark scores:
 - NPB: Mop/s (Millions of Operations per Second)
 - Unixbench: Loops per second (lps), KB/sec, and operations per second

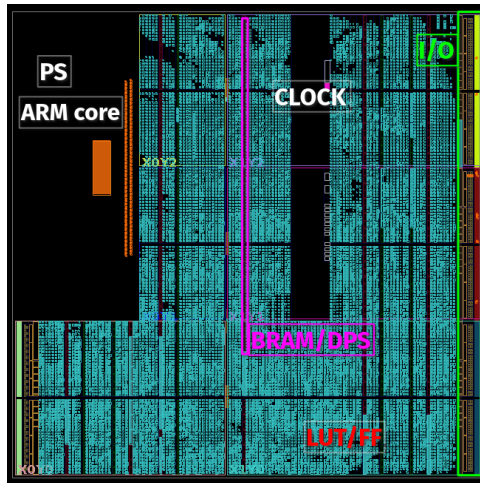
How do We Measure Performance and Resource Usage?

Experimental Setup



CVA6 *Single Core* deployment on a PYNQ-Z2 FPGA with PMod Modules

How do We Measure Resource Usage?

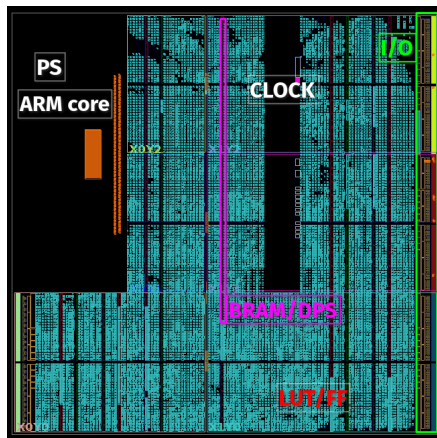


Zynq-7000 SoC

The SoC (System on a chip) is mapped onto the FPGA's programmable logic, utilizing the following hardware resources to implement the CVA6 design:

- Look Up Tables (**LUT**) for combinatorial logic.
- Flip Flops (**FF**) for state storage
- Block Random Access Memory (**BRAM**) for fast on-chip memory
- Digital Signal Processor (**DSP**) for high-performance math calculations

How do We Measure Resource Usage?



FPGA Area

- LUT
- FF
- DSP
- BRAM

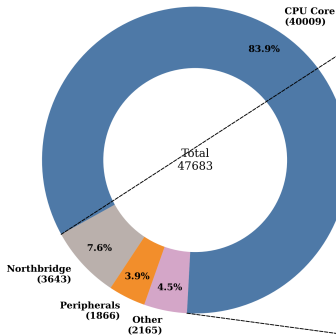
CVA6 stages

- Fetch
- Decode
- Issue
- Execute
- Memory
- Commit

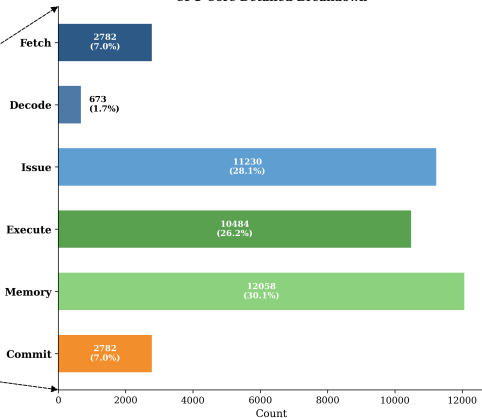
How do We Measure Resource Usage?

Initial LUT Usage Measurement

SoC Breakdown: LUT

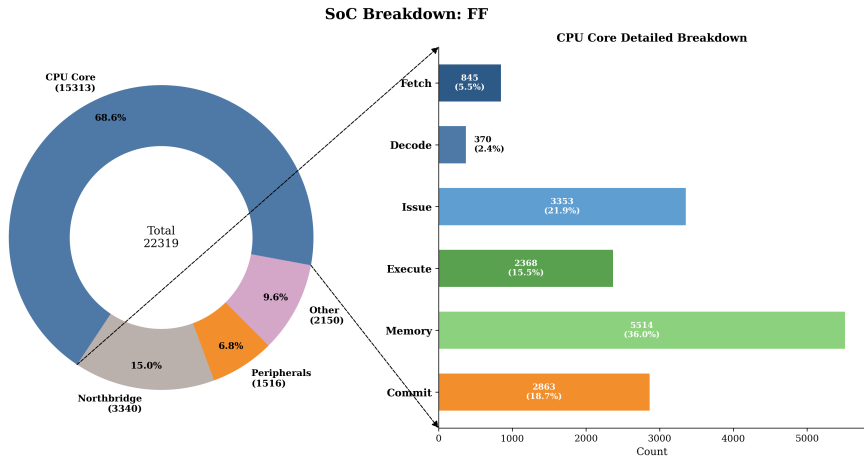


CPU Core Detailed Breakdown



How do We Measure Resource Usage?

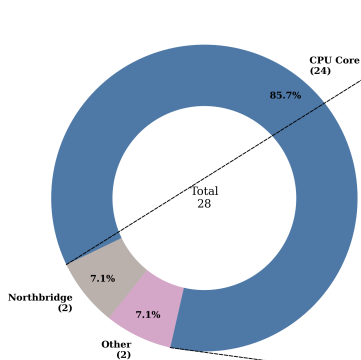
Initial FF Usage Measurement



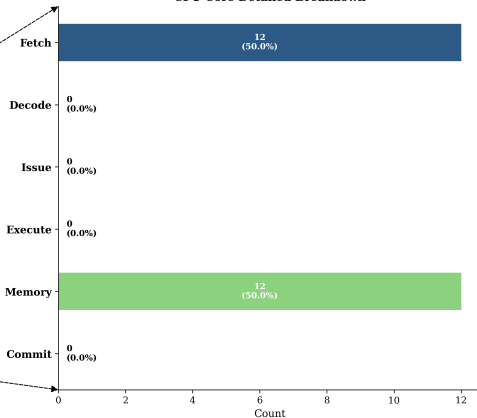
How do We Measure Resource Usage?

Initial BRAM Usage Measurement

SoC Breakdown: BRAM

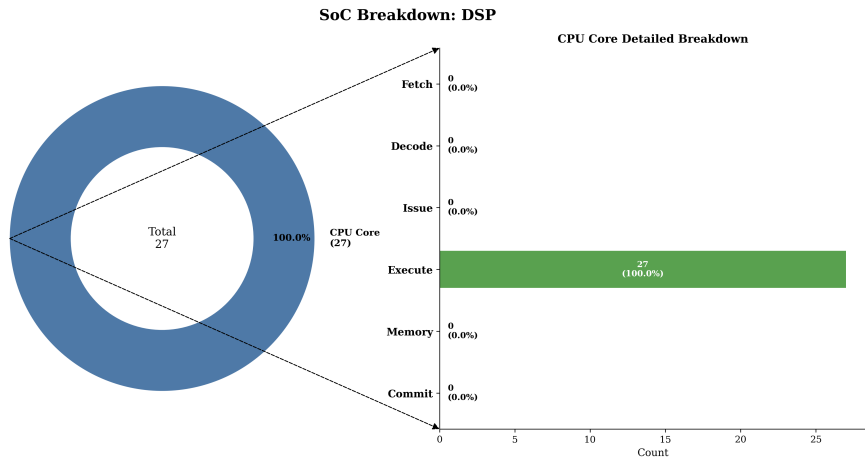


CPU Core Detailed Breakdown



How do We Measure Resource Usage?

Initial DSP Usage Measurement



Outline

- 1 Motivation
- 2 Performance and Resource Usage Measurement
- 3 Experiments**
- 4 Results
- 5 Contributions
- 6 Conclusion and Future work

Baseline Architecture

To study the effects of the micro-architectural parameters the following baseline parameters were used.

Parameter	Size	CVA6 Stage
Data Cache	4096[bytes]	Memory
Data Cache Associativity	4	Memory
Data Cache Line Width	128	Memory
Instruction Cache	4096[bytes]	Fetch
Instruction Cache Associativity	4	Fetch
Instruction Cache Line Width	128	Fetch
Branch Target Buffer (BTB) Entries	16	Fetch
Branch History Table (BHT) Entries	16	Fetch
Retirement Width of the CPU	2	Commit

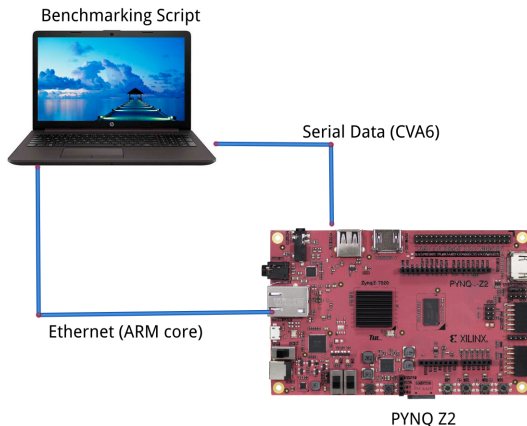
Parameter Sweeps

And the following parameters sweeps were proposed to study their effect on performance and resource utilization.

Sweep	Range	CVA6 Stage
Data Cache	128[bytes] $\rightarrow$ 256k[bytes]	Memory
Data / Instruction Cache Associativity	1 $\rightarrow$ 16	Memory / Fetch
Instruction Cache	2k[bytes] $\rightarrow$ 256k[bytes]	Fetch
BTB Entries	8 $\rightarrow$ 32k	Fetch
BHT Entries	16 $\rightarrow$ 16k	Fetch
Retirement Width of the CPU	2 $\rightarrow$ 1	Commit

Automated Benchmarking Framework

To automate design space exploration, we developed a framework which uses the ARM core and the CVA6 core, the full run of the benchmarks for all the parameter sweeps takes 21 hours.

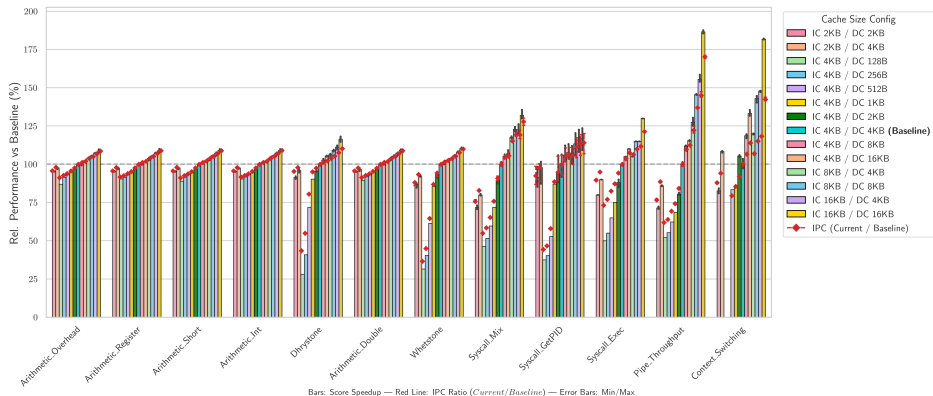


Outline

- 1 Motivation
- 2 Performance and Resource Usage Measurement
- 3 Experiments
- 4 Results**
- 5 Contributions
- 6 Conclusion and Future work

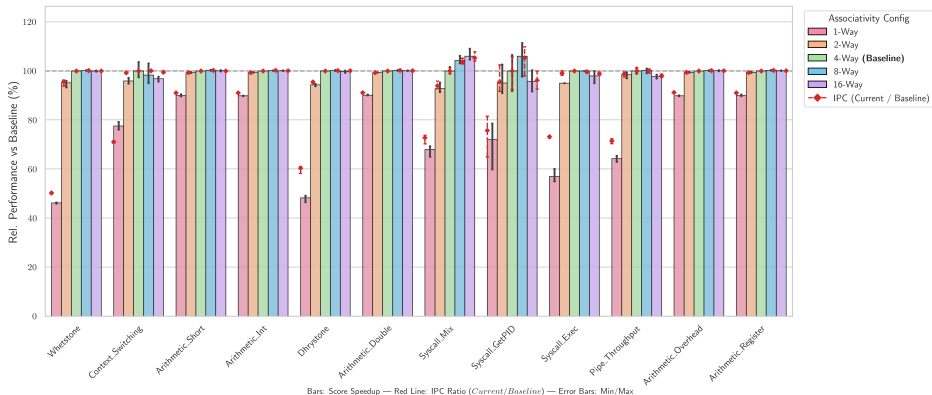
Performance Dynamics

UnixBench Suite: Sensitivity to Cache Size



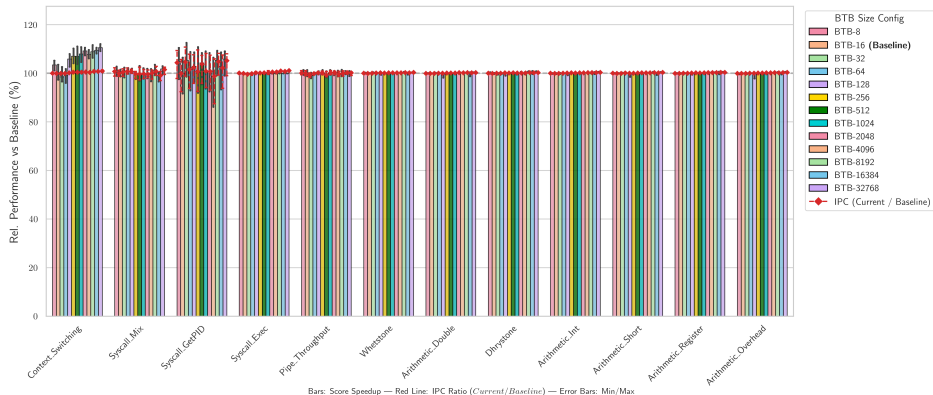
Performance Dynamics

UnixBench Suite: Sensitivity to Associativity



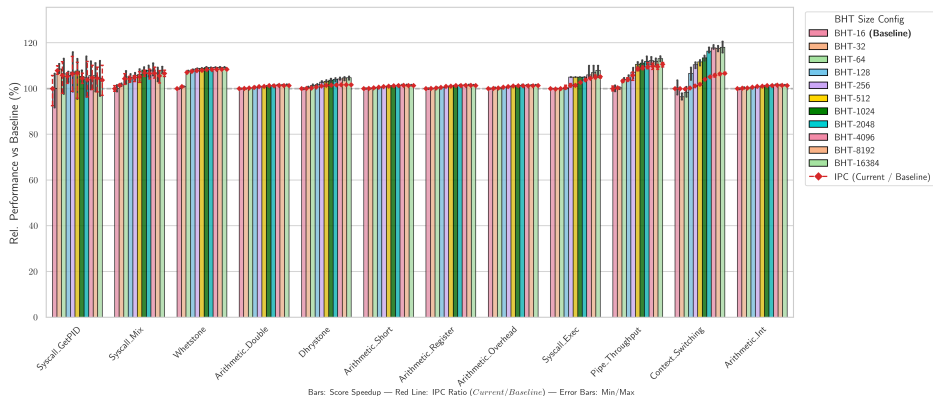
Performance Dynamics

UnixBench Suite: Sensitivity to BTB Size

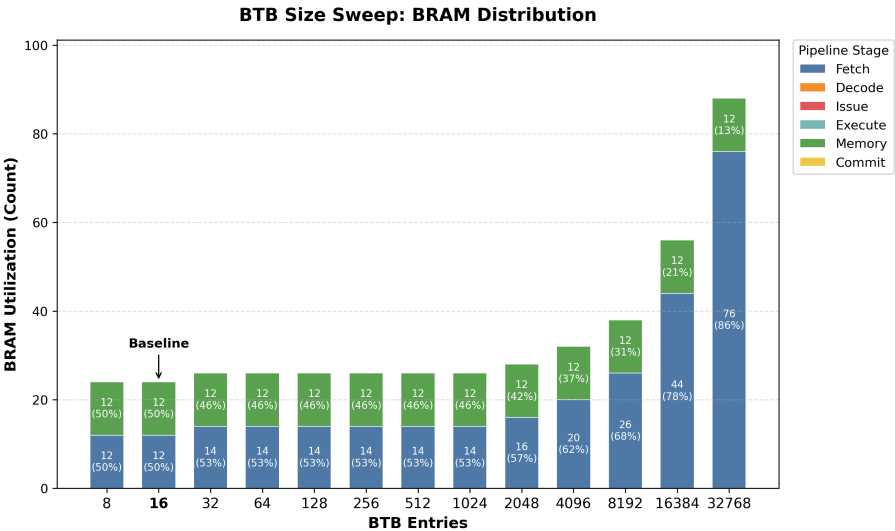


Performance Dynamics

UnixBench Suite: Sensitivity to BHT Size

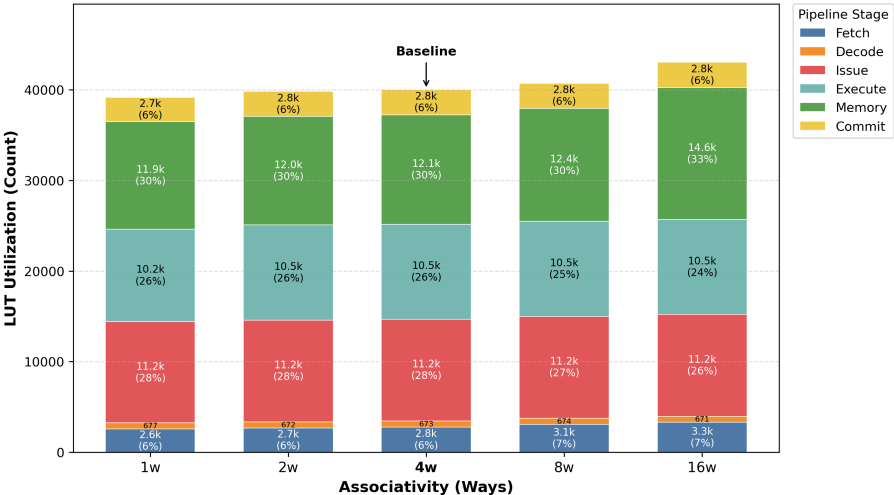


Resource Usage Dynamics



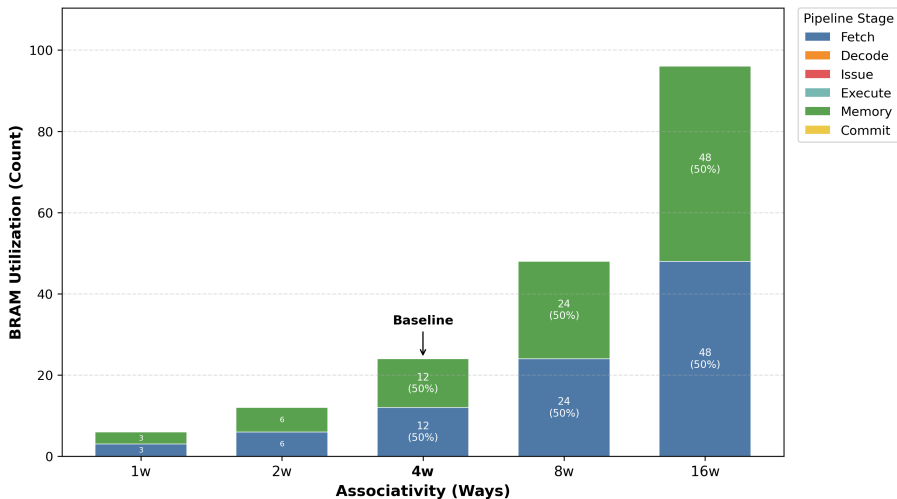
Resource Usage Dynamics

Associativity Sweep: LUT Distribution



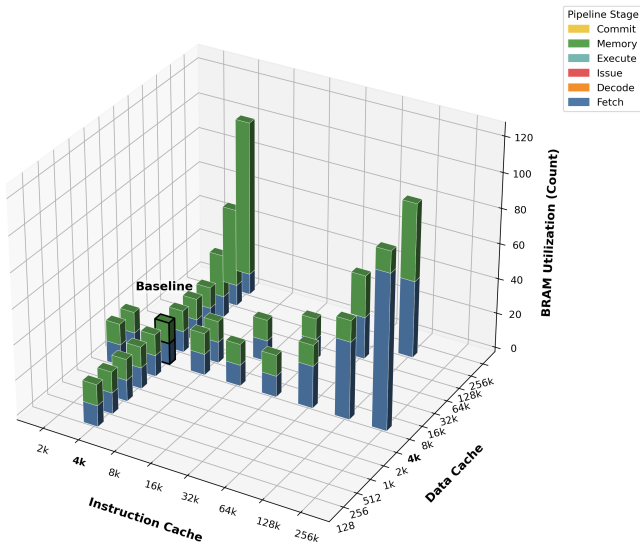
Resource Usage Dynamics

Associativity Sweep: BRAM Distribution



Resource Usage Dynamics

Cache Sweep: BRAM Distribution



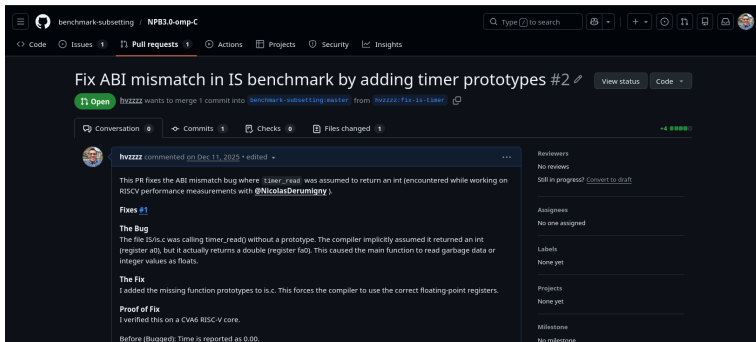
Outline

- 1 Motivation
- 2 Performance and Resource Usage Measurement
- 3 Experiments
- 4 Results
- 5 Contributions**
- 6 Conclusion and Future work

Contributions

- PR to the NPB repository

<https://github.com/benchmark-subsetting/NPB3.0-omp-C/pull/2>



- Automated tool for benchmarking architectural changes on the PYNQ Z2
- Baseline performance / utilization measurements for CVA6 multi-core

Outline

- 1 Motivation
- 2 Performance and Resource Usage Measurement
- 3 Experiments
- 4 Results
- 5 Contributions
- 6 Conclusion and Future work

Conclusion

- *Automated Benchmarking Established*: Successfully developed an automated evaluation tool for the PYNQ-Z2 , securing complete single-core performance and resource baselines.
- *Cache Size Trade-offs*: Larger instruction and data caches significantly accelerate workloads like context switching and system calls , though they require substantially more BRAM.
- *Inefficient Scaling Returns*: Scaling cache associativity and branch predictors (BTB/BHT) yields negligible performance improvements while heavily increasing LUT and BRAM consumption.
- *Prepared for Multi-Core Expansion*: The established baselines clear the path for the next project phase: developing the shared resource arbitrator and decoupling the core.

Future Work

Stage 4 (Core Modification & Decoupling)

Development
of a modified
CVA6 version

Implementation
of an external
module
interface [optional]

Enabling
connection for
FPU
management [optional]

Stage 5 (Arbitration Module Design)

Development
of the shared
resource
arbitrator

Implementation
of pipeline
stalling logic [optional]

Routing of
results
between
multiple cores [optional]

Stage 6 (System Integration & Evaluation)

Evaluation of
performance
and FPGA
utilization
(FF/LUT/BRAM)

Evaluation of
performance
NPB and
Unixbench

Thank you!

hanan.quispecondori@ip-paris.fr

This work is being supported by the **BENAGIL TEAM**

The Inria logo is written in a stylized, red, cursive script.